

Değerlendirme Metrikleri: Kapsamlı Teknik Açıklama

VAE ve Dense Autoencoder ile Ağ Saldırısı Tespitinde Kullanılan
Tüm Metriklerin Matematiksel Mantığı ve Projenin Kendi
Sonuçlarıyla Bağlantısı

Doküman: METRIKLER_ACIKLAMA.md

Proje: IDS-Project / 10_final_report

Tarih: 28 Temmuz 2026

Kapsam: Confusion Matrix, Recall, Precision, F1,
ROC-AUC, PR-AUC, KS İstatistiği, Bootstrap CI,
Reconstruction Error

İçindekiler

Giriş: Bu Doküman Ne İçin Var

0. Temel Kavram: Confusion Matrix (Karışıklık Matrisi)
 1. Recall (Attack Recall / Sensitivity / True Positive Rate)
 2. Precision
 3. F1 Score
 4. ROC-AUC (Receiver Operating Characteristic — Area Under Curve)
 5. PR-AUC (Precision-Recall Curve — Area Under Curve)
 6. KS İstatistiği (Kolmogorov-Smirnov Testi)
 7. Bootstrap Confidence Interval (Bootstrap Güven Aralığı)
 8. Reconstruction Error (Yeniden İnşa Hatası)
- Özet Tablo: Hangi Metriğe Ne Zaman Bakmalı

Giriş: Bu Doküman Ne İçin Var

Bu proje, ağ trafiğindeki saldırı akışlarını (apache_bench, portscan, slowloris) benign (normal) trafikten ayırt etmeye çalışan iki unsupervised anomali tespit modeli (VAE ve Dense Autoencoder) değerlendiriyor. Bu değerlendirmeler tek bir sayıyla ("model %90 doğru") özetlenemeyecek kadar nüanslı — çünkü **dengesiz bir veri setinde** ("imbalanced": benign flow sayısı attack flow sayısından kat kat fazla), **hangi hatanın** (saldırıyı kaçırmak mı, yanlış alarm vermek mi) daha maliyetli olduğuna göre farklı metrikler tamamen farklı hikayeler anlatabiliyor.

Bu doküman, projede kullanılan her metriği üç seviyede açıklıyor:

1. **Matematiksel tanım** — formül ve confusion matrix ile ilişkisi.
2. **NIDS bağlamındaki anlamı** — bu metrik hangi hatayı cezalandırıyor, bir güvenlik ekibi için ne ifade ediyor.
3. **Projenin kendi sonuçlarıyla somut bağlantı** — gerçek sayılarımız
(10_final_report/01_single_attack_type/ , 04_apache_bench_diagnostics/ ,
phase3_vae/05_contamination_sweep/) üzerinden.

Amaç, bu dosyayı okuyan birinin hem "bu metrik ne anlama geliyor" hem de "projede bu metrik neden bu sayıyı verdi, ne anlatıyor" sorularının ikisine de cevap bulabilmesi.

Kullanılan modeller (referans için):

- **VAE** (Variational Autoencoder): latent boyut = 10, beta = 0.25, clean-only (%0 kontaminasyon) eğitilmiş, 20 farklı ağırlık-başlangıç seed'i ile değerlendirildi.
- **Dense v1** (Dense Autoencoder): full_features varyantı, 5 seed.
- Her iki model de reconstruction-error tabanlı: eğitim sadece benign trafik üzerinde yapılıyor, model "normal"i öğreniyor, test zamanında bir flow'un reconstruction (yeniden inşa) hatası ne kadar yüksekse o kadar "anormal" (potansiyel saldırı) sayılıyor.
- Sabit eşik (threshold): held-out benign validasyon setinin reconstruction-error dağılımının **95. percentile**'i (threshold_95) — yani "benign trafiğin en kötü %5'inin üzerinde kalan her şey saldırı alarmı" kuralı.

0. Temel Kavram: Confusion Matrix (Karışıklık Matrisi)

Her ikili (binary) sınıflandırma kararı — burada "bu flow saldırı mı, benign mi?" — dört olası sonuca ayrılır. "Pozitif" sınıf bu projede **saldırı (attack)**, "negatif" sınıf **benign**.

	Model "saldırı" dedi (Pozitif tahmin)	Model "benign" dedi (Negatif tahmin)
Gerçekte saldırı	TP (True Positive) — doğru yakalanan saldırı	FN (False Negative) — kaçırılan saldırı
Gerçekte benign	FP (False Positive) — yanlış alarm	TN (True Negative) — doğru tanınan normal trafik

Bu dört sayı, aşağıda anlatılan hemen hemen tüm metriklerin ham malzemesidir. NIDS bağlamında iki hata türünün maliyeti çok farklıdır:

- **FN (kaçırılan saldırı):** bir saldırgan tespit edilmeden sisteme giriyor. Güvenlik açısından genellikle **en pahalı** hata.
- **FP (yanlış alarm):** normal bir kullanıcı işlemi saldırı olarak işaretleniyor. Güvenlik ekibinin zamanını tüketiyor ("alarm yorgunluğu" / alert fatigue), çok fazla FP olursa gerçek alarmlar da göz ardı edilmeye başlar.

Projenin her sonucu — VAE'nin apache_bench'i neden kaçırdığından (04_apache_bench_diagnostics/), kontaminasyonun modelin bu dengeyi nasıl bozduğuna (phase3_vae/05_contamination_sweep/) kadar — bu dört sayının farklı kombinasyonlarını okumaktan ibaret.

1. Recall (Attack Recall / Sensitivity / True Positive Rate)

Formül/tanım:

$$\text{Recall} = \frac{TP}{TP + FN}$$

Sade dille: *gerçekte var olan tüm saldırılardan kaçını yakaladık?* $TP+FN$, veri setindeki **toplam gerçek saldırı sayısı**dır (model ne derse desin) — yani recall'ün paydası saf gerçekliktir, modelin tahminlerinden bağımsızdır.

Ne soruyor: "100 saldırı varsa, kaçını kaçırmadık?"

Confusion matrix ile ilişkisi: Sadece "gerçekte saldırı" satırına bakar (TP ve FN). FP ve TN'yi (benign tarafını) hiç hesaba katmaz — recall'ün ne kadar yanlış alarm verildiğiyle hiçbir ilgisi yoktur.

NIDS bağlamında neden önemli: Recall doğrudan **kaçırma (FN)** hatasını cezalandırır. Bir NIDS'in en temel görevi saldırıları yakalamaktır; düşük recall, sistemin saldırganlara karşı gerçekte ne kadar "kör" olduğunu gösterir. Güvenlik literatüründe genelde en kritik metrik budur çünkü bir tek kaçırılan saldırı (örn. bir veri sızıntısı) yüzlerce yanlış alarmdan daha pahalıya mal olabilir.

Projedeki somut sonuç: `01_single_attack_type/vae/results.md` 'den (clean-only VAE, threshold_95, 20 seed ortalaması):

attack_type	attack recall (thr95)	Yorum
apache_bench	0.0328 (±0.0055)	1487 saldırıdan sadece ~49'u yakalanıyor, ~1438'i (%96.7) kaçıyor
portscan	0.9889 (±0.0138)	694 saldırıdan ~686'sı yakalanıyor, sadece ~8'i kaçıyor
slowloris	1.0000 (±0.0000)	929 saldırının tamamı, her seed'de, kusursuzca yakalanıyor

apache_bench'in recall'ü 0.0328 — yani model bu saldırı türünü pratikte **görmezden geliyor**. `03_segmented_injection/vae/block_recall_f1.md` bu sonucun saldırının test akışındaki konumundan (karışık mı, bitişik blok mu) bağımsız olduğunu doğruluyor: bloklu enjeksiyonda da recall 0.0322 — neredeyse birebir aynı. Bu, `04_apache_bench_diagnostics/` klasöründeki kök-neden analizinin çıkış noktasıdır.

Güçlü/zayıf yönleri: Recall tek başına yanıltıcı olabilir çünkü **eşiği sonsuza kadar düşürerek** (her şeyi "saldırı" diye işaretleyerek) recall'ü kolayca 1.0'a çıkarabilirsiniz — bu, modelin hiçbir ayırt etme gücü olmadan da mümkündür (bkz. ROC-AUC = 0.5 durumu, aşağıda). Recall'ü her zaman **Precision** ile birlikte okumak gerekir: yüksek recall, düşük precision ile geliyorsa, sistem "her şeyi alarma çeviren" işe yaramaz bir dedektöre dönüşmüş olabilir.

Diğer metriklerle ilişkisi/gerilimi: Recall ve Precision klasik bir **trade-off** (ödünlüşim) içindedir — eşiği düşürdükçe recall artar ama genelde precision düşer (daha çok benign flow da "saldırı" diye işaretlenmeye başlar). Bu projede eşik sabit tutulduğu (threshold_95) için bu trade-off'u doğrudan gözlemlemiyoruz, ama

contamination sweep deneyinde (`phase3_vae/05_contamination_sweep/`) benzer bir dinamik görülüyor: kontaminasyon arttıkça eşiğin kendisi büyüyor (thr95, %0'da ~0.11-0.15 iken %12'de ~0.37-1.27'ye çıkıyor) ve model "daha toleranslı" hale geldikçe attack recall de oynaklaşıyor.

2. Precision

Formül/tanım:

$$\text{Precision} = \frac{TP}{TP + FP}$$

Sade dille: "saldırı" dediğimiz her şeyden kaç gerçekten saldırıydı? TP+FP, modelin **pozitif tahmin ettiği her şeydir** (doğru olsun olmasın) — yani precision'ın paydası modelin kendi kararlarıdır, gerçeklikten değil.

Ne soruyor: "Alarm çaldığımızda, bu alarma ne kadar güvenebiliriz?"

Confusion matrix ile ilişkisi: Sadece "model pozitif tahmin etti" sütununa bakar (TP ve FP). Kaçırılan saldırıları (FN) hiç hesaba katmaz — bir model hiç alarm vermese bile (recall=0), verdiği tek tük alarmların hepsi doğruysa precision=1.0 olabilir.

NIDS bağlamında neden önemli: Precision doğrudan **yanlış alarm (FP)** hatasını cezalandırır. Düşük precision, güvenlik ekibinin her gün onlarca sahte alarmı elemek zorunda kalması demektir — bu hem operasyonel maliyet hem de "alert fatigue" yaratır: ekip zamanla alarmlara güvenmeyi bırakır ve gerçek bir saldırı da gözden kaçabilir. Precision, sistemin **pratikte kullanılabilir olup olmadığını** ölçer.

Projedeki somut sonuç: Proje çıktılarında precision doğrudan tabloya yazılmamış (F1 ve recall raporlanıyor), ama F1'in tanımından (bkz. Bölüm 3) geriye doğru hesaplanabilir: $P = \frac{F1 \cdot R}{2R - F1}$. Bu formülle 01_single_attack_type/vae/results.md 'deki sayılardan:

attack_type	F1 (thr95)	Recall (thr95)	Türetilmiş Precision
apache_bench	0.0507	0.0328	~0.112
portscan	0.7737	0.9889	~0.635
slowloris	0.8271	1.0000	~0.705

Bu, apache_bench için özellikle çarpıcı bir sonuç: model saldırıların sadece %3.3'ünü yakalıyor (çok düşük recall) **ve** verdiği o az sayıdaki alarmın da sadece ~%11'i gerçek apache_bench saldırısı (düşük precision) — yani model bu saldırı türünde hem kör hem de güvenilmez. Buna karşılık portscan ve slowloris'te precision ~%64-70 civarında: recall neredeyse mükemmel olsa da, benign trafiğin sabit ~%5.7-5.8 FPR'si (bkz. Bölüm 0/1) precision'ı 1.0'dan aşağı çekiyor, çünkü test setinde benign sayısı (6821) attack sayısından (694-929) çok daha fazla — dengesiz veri setinde precision'ın FP'lere karşı ne kadar hassas olduğunun somut bir örneği.

Güçlü/zayıf yönleri: Precision, **sınıf dengesizliğinden çok etkilenir**. Aynı FPR (benign'in yanlış alarma çevrilme oranı) sabit kalsa bile, test setindeki benign/attack oranı değiştikçe precision değeri kayar — çünkü FP sayısı mutlak olarak benign popülasyonu ile orantılıdır. Bu yüzden precision tek başına "modelin gücü" hakkında taşınabilir bir sinyal değildir; aynı modelin farklı ortamlarda (farklı attack/benign oranlarında) çok farklı precision'lar vermesi normaldir. Recall ile birlikte okunmalı.

Diğer metriklerle ilişkisi/gerilimi: Precision-Recall trade-off'unun diğer yarısı. Eşiği yükseltirseniz (daha "muhafazakâr" model — sadece çok net anomalilerde alarm ver) precision genelde artar ama recall düşer. Dengesiz veri setlerinde bu trade-off'u **eşikten bağımsız** biçimde özetleyen araç PR-AUC'tur (bkz. Bölüm 5).

3. F1 Score

Formül/tanım:

$$F1 = 2 \cdot \frac{Precision \cdot Recall}{Precision + Recall} = \frac{2TP}{2TP + FP + FN}$$

Precision ve Recall'ün **harmonik ortalaması**.

Neden aritmetik ortalama değil, harmonik ortalama: Aritmetik ortalama $\frac{P+R}{2}$ kullansaydık, biri çok yüksek biri çok düşükken bile "iyi" bir skor çıkabilirdi — örneğin $P=1.0$, $R=0.02$ için aritmetik ortalama 0.51 (orta karar gibi görünür) ama gerçekte model neredeyse hiçbir şeyi yakalamıyor. Harmonik ortalama, **küçük olan değere çok daha fazla ağırlık verir** — aynı örnekte $F1 = 2 \cdot (1.0 \cdot 0.02) / (1.0 + 0.02) \approx 0.039$, yani gerçek durumu (model işe yaramaz derecede az yakalıyor) çok daha doğru yansıtır. Harmonik ortalama matematiksel olarak, iki sayıdan **küçük olanına** yakın durur; F1'in "her iki metrik de makul olmadıkça yüksek çıkmama" özelliği buradan gelir.

Ne soruyor: "Precision ve recall'ü **tek bir sayıda**, ikisi de kötü olduğunda cezalandıracak şekilde nasıl özetlerim?"

Confusion matrix ile ilişkisi: TP, FP, FN'nin üçünü birden kullanır (TN'i kullanmaz — yani "doğru tanınan benign" sayısını hiç görmez, bu da F1'i dengesiz veri setlerinde TN'in domine ettiği durumlarda accuracy'den daha anlamlı kılar).

NIDS bağlamında neden önemli: F1, ne kaçırma (FN) ne de yanlış alarm (FP) hatasına özel bir öncelik vermeden **ikisini dengeli** biçimde cezalandırmak istediğinizde kullanılır — tek bir sayıyla model karşılaştırması/sıralaması yapmak gerektiğinde pratik bir özet metriktir.

Projedeki somut sonuç: 01_single_attack_type/vae/results.md :

attack_type	F1 (thr95)
apache_bench	0.0507 (± 0.0081)
portscan	0.7737 (± 0.0161)
slowloris	0.8271 (± 0.0158)

apache_bench'in F1'i 0.05 — hem düşük recall (0.033) hem düşük precision (~ 0.11) olduğu için harmonik ortalama bunu acımasızca yansıtır: aritmetik ortalama olsaydı $(0.033 + 0.112) / 2 \approx 0.073$ çıkardı, harmonik ortalama daha da düşük (0.0507) çıkıyor çünkü her iki bileşen de zayıf.

Güçlü/zayıf yönleri: F1, precision ve recall'e **eşit ağırlık** verir — ama NIDS'te bu iki hatanın maliyeti nadiren eşittir (genelde FN daha pahalıdır). F1 yüksek çıksa bile, hangi hatanın (FP mi FN mi) daha baskın olduğunu göstermez; bu yüzden F1'i **her zaman precision ve recall'ün kendileriyle birlikte** raporlamak (tıpkı bu projenin tablolarının yaptığı gibi) gerekir — tek başına F1, "model iyi mi kötü mü" sorusuna kısmi bir cevap verir.

Diğer metriklerle ilişkisi/gerilimi: F1, sabit bir eşikteki (bu projede threshold_95) tek bir precision-recall noktasını özetler. Eşik değıştikçe F1 de değışir — bu yüzden "eşikten bağımsız" bir karşılaştırma için PR-AUC'a (Bölüm 5) bakmak gerekir. F2 skoru (bu projede bazı ara script'lerde hesaplanan, F1'in recall'e daha fazla ağırlık veren bir varyantı) NIDS'te FN'in FP'den daha pahalı olduđu durumlar için tercih edilebilir, ama bu dokümandaki ana raporlarda F1 kullanılmıştır.

4. ROC-AUC (Receiver Operating Characteristic — Area Under Curve)

Formül/tanım:

ROC eğrisi, **her olası eşik değerinde** iki oranı birbirine karşı çizer:

- Y eksenini: **True Positive Rate (TPR)** = Recall = $TP / (TP + FN)$
- X eksenini: **False Positive Rate (FPR)** = $FP / (FP + TN)$

Eşiği en düşükten en yükseğe tararken (her şeyi "saldırı" demekten hiçbir şeyi "saldırı" dememeye) bu (FPR, TPR) noktalarının çizdiği eğri ROC eğrisidir. **ROC-AUC**, bu eğrinin altında kalan alandır (0 ile 1 arası).

Eşdeğer bir yorum: ROC-AUC, rastgele seçilmiş bir pozitif (saldırı) örneğin, rastgele seçilmiş bir negatif (benign) örnekten **daha yüksek anomali skoru alma olasılığıdır**.

Ne soruyor: "Model, tüm olası eşik seçimleri boyunca ortalamada, saldırıları benign'den ne kadar iyi ayırt edebiliyor?" — **eşikten bağımsız** bir ayırt etme gücü (discriminative power) ölçüsü.

Confusion matrix ile ilişkisi: Tek bir confusion matrix'ten değil, her olası eşikteki confusion matrix'ten (yani TPR/FPR çiftinden) inşa edilir. threshold_95 gibi tek bir sabit eşiğe bağlı değildir.

0.5 = rastgele, 1.0 = mükemmel: ROC-AUC=0.5, modelin bir saldırıyla bir benign'i ayırt etme gücünün **yazı-tura atmaktan farksız** olduğu anlamına gelir (rastgele sıralanmış bir skor listesi de ortalamada 0.5 verir). ROC-AUC=1.0, her saldırının her benign'den daha yüksek skor aldığı, yani **mükemmel ayırma** demektir. 0.5'in altı, modelin skorunun ters çevrilmiş olması gerektiği (sistemik ama ters yönlü bir sinyal) anlamına gelir, ki bu proje sonuçlarında görülüyor.

NIDS bağlamında neden önemli: ROC-AUC, belirli bir eşik seçmeden **"bu feature seti/model, bu saldırı türünü prensipte ayırt edebiliyor mu?"** sorusuna cevap verir. Düşük ROC-AUC, sorunun eşik ayarında değil, modelin/feature'ların o saldırı türü için **hiç sinyal taşımamasında** olduğunu gösterir — bu, eşiği ne kadar ince ayarlarsanız ayarlayın düzelmeyecek temel bir sorundur.

Projedeki somut sonuç: apache_bench için VAE ROC-AUC = **0.5815** (± 0.0768) — 0.5'e (rastgele tahmin) çok yakın. Bu, projenin en önemli negatif bulgularından biri ve `04_apache_bench_diagnostics/findings.md` dosyasının tamamı bu sonucun **neden** böyle çıktığını araştırıyor. Karşılaştırma: portscan ROC-AUC = 0.9982, slowloris ROC-AUC = 1.0000 — neredeyse kusursuz ayırma.

`04_apache_bench_diagnostics/` bulgusu şu: apache_bench'in tekil-flow feature'ları (paket sayısı, byte hacmi, süre) benign'den **istatistiksel olarak anlamlı** şekilde farklı (KS istatistikleri 0.62-0.76, $p \approx 0$) ama bu farkın **büyüklüğü** küçük — apache_bench'in ortalaması benign ortalamasından sadece ~0.4-0.7 benign standart sapması uzakta, yani hâlâ benign'in "normal" aralığının **içinde**. ROC-AUC=0.58 tam olarak bunu yansıtıyor: istatistiksel olarak ayrılabilir ama pratikte, tek bir flow'un skoruna bakarak, ayırt etmek neredeyse

imkânsız — bu yüzden hem düşük ROC-AUC hem düşük ROC-AUC'un beklediği gibi düşük recall (Bölüm 1) bir arada görülüyor.

Güçlü/zayıf yönleri: ROC-AUC, **dengeşiz veri setlerinde yanıltıcı derecede iyimser** olabilir çünkü FPR'nin paydası ($FP + TN$) benign sayısı ile domine edilir — benign sayısı çok büyükse, FP sayısında büyük mutlak artışlar bile FPR'yi çok az deęiştirir, bu da eğrinin sol tarafını "yapay olarak iyi" gösterebilir. Bu projede attack/benign oranı dengeşiz olduęu için (6821 benign'e karşı 694-1487 attack), ROC-AUC tek başına yeterli deęildir — bu yüzden PR-AUC (Bölüm 5) paralel olarak raporlanıyor.

Dięer metriklerle ilişkisi/gerilimi: ROC-AUC ile PR-AUC genelde aynı yönde hareket eder (apache_bench'te ikisi de düşük: 0.58 / 0.21) ama farklı bir ölçekte — bu farkın önemi tam olarak dengeşiz veri setlerinde ortaya çıkar (bkz. Bölüm 5). ROC-AUC, F1/recall/precision'ın aksine **tek bir eşięe baęlı deęildir** — modelin "potansiyelini" gösterir, ama pratikte kullanılacak sabit bir eşikte (threshold_95) o potansiyelin ne kadarının geręekleştiğini göstermez; onun için F1/recall/precision tablolarına bakmak gerekir.

5. PR-AUC (Precision-Recall Curve — Area Under Curve)

Formül/tanım:

PR eğrisi, eşiği tararken Precision'ı ($TP/(TP + FP)$) Recall'a ($TP/(TP + FN)$) karşı çizer. **PR-AUC**, bu eğrinin altında kalan alan.

Ne soruyor: "Model, farklı recall seviyelerinde (ne kadar saldırıyı yakalamaya çalışırsak çalışalım), ne kadar precision koruyabiliyor?" — yani hem kaçırma hem yanlış alarm dengesini, eşikten bağımsız biçimde özetler.

ROC-AUC'dan farkı: ROC-AUC'un FPR ekseni ($FP/(FP + TN)$) paydasında **TN** (doğru tanınan benign) vardır — TN sayısı çoğunlukla çok büyük olduğu için FPR "gürbüz" (robust) kalır ve az sayıda FP eklenmesi FPR'yi pek değiştirmez. PR eğrisinin Precision ekseni ($TP/(TP + FP)$) ise **hiç TN içermez** — paydası sadece modelin pozitif dediği şeylerdir. Bu yüzden az sayıda benign örnek bile yanlışlıkla "saldırı" damgası yerse, bu Precision'ı doğrudan ve sertçe düşürür; TN sayısının büyüklüğü onu seyreletmez.

Dengesiz veri setlerinde neden PR-AUC daha güvenilir: Bu projede benign sayısı (6821) attack sayısından (apache_bench: 1487, portscan: 694, slowloris: 929) kat kat fazla — yani TN potansiyel olarak çok büyük bir sayı. Böyle bir ortamda ROC-AUC, gerçekte önemli sayıda FP üretilse bile (çünkü FP, dev TN havuzunun yanında küçük bir oran gibi görünür) yüksek çıkabilir — "iyimser" bir tablo çizer. PR-AUC, TN'yi hiç görmediği için bu seyreletme etkisinden muaf: precision'ı gerçekten düşüren her FP, PR eğrisini doğrudan aşağı çeker. Bu yüzden dengesiz veri setlerinde (attack azınlıkta olduğunda) PR-AUC, modelin **pratik** kullanılabilirliğini ROC-AUC'dan daha dürüst yansıtır.

Projedeki somut sonuç: apache_bench için VAE PR-AUC = **0.2133** (± 0.0219) — ROC-AUC'un (0.5815) aksine, bu sayı "rastgeleye yakın" bir referans noktasına sahip değildir çünkü PR-AUC'un rastgele-model taban çizgisi sınıf oranına eşittir: bu alt-küme için attack oranı $1487/(1487+6821) \approx 0.179$, yani **rastgele bir model burada PR-AUC ≈ 0.179 verirdi**. VAE'nin 0.2133'ü bu taban çizgisine (0.179) yakın — ROC-AUC'un "0.58, rastgeleye (0.5) yakın ama biraz üstünde" izlenimini teyit ediyor, üstelik dengesiz-veri seyreletmesinden arınmış biçimde. Karşılaştırma: portscan PR-AUC = 0.9886, slowloris PR-AUC = 1.0000 — kendi sınıf oranlarına (sırasıyla ~ 0.092 ve ~ 0.120) göre inanılmaz yüksek, yani gerçek ayrışma.

Contamination sweep deneyinde (`phase3_vae/05_contamination_sweep/`) de ana metrik olarak **PR-AUC** seçilmiş olması tesadüf değil — o deneyde de sabit test setinde attack oranı sadece %10.04 (73 attack / 654 benign), yani dengesiz; PR-AUC bu yüzden ROC-AUC yerine "birincil" metrik olarak kullanılıyor (`bootstrap_significance.py` , `METRIC = "pr_auc"`).

Güçlü/zayıf yönleri: PR-AUC'un rastgele-taban-çizgisi sınıf oranına bağlı olduğu için, **farklı sınıf oranlarına sahip veri setleri arasında PR-AUC'ları doğrudan karşılaştırmak yanıltıcıdır** (0.21 "kötü" mü "iyi" mi, hangi taban çizgisine göre baktığınıza bağlı) — her zaman o alt-kümenin kendi rastgele-taban-çizgisiyle (sınıf oranıyla) karşılaştırılmalı. Bu proje bunu doğal olarak yapıyor çünkü her tablo $n_{\text{attack}}/n_{\text{benign}}$ sayılarını da raporluyor.

Diğer metriklerle ilişkisi/gerilimi: PR-AUC, F1'in "eşikten bağımsız, tüm eşikler boyunca" hâli gibi düşünülebilir — F1 tek bir (precision, recall) noktasını özetlerken, PR-AUC o eğrinin tamamının altındaki alanı özetler. ROC-AUC ile PR-AUC genelde aynı yönde hareket eder ama dengesiz veri setlerinde büyüklükleri arasındaki fark (apache_bench'te 0.58 vs 0.21) tam olarak yukarıda açıklanan TN-seyreltme etkisinin bir göstergesidir.

6. KS İstatistiği (Kolmogorov-Smirnov Testi)

Formül/tanım:

$$D = \sup_x |F_1(x) - F_2(x)|$$

İki örneklemin **ampirik kümülatif dağılım fonksiyonları** (CDF) arasındaki **en büyük dikey mesafe**. F_1 ve F_2 , sırasıyla iki grubun (örn. benign ve apache_bench) bir feature üzerindeki kümülatif dağılım fonksiyonlarıdır. D, 0 (dağılımlar tamamen örtüşüyor) ile 1 (dağılımlar tamamen ayrık, hiç örtüşmüyor) arasında bir değer alır. Eşlik eden p-değeri, "bu iki örneklem aynı dağılımdan geliyor" sıfır hipotezini test eder.

Ne soruyor: "Bu iki grup (örn. benign vs apache_bench flow'ları), bu feature'da **istatistiksel olarak** farklı dağılımlara mı sahip?" — dikkat: bu, "farkın **büyüklüğü** pratikte önemli mi?" sorusuyla aynı şey değildir (bkz. aşağıdaki paradoks).

Projede nasıl kullanıldı: `04_apache_bench_diagnostics/findings.md`, apache_bench'in her feature'ında (18 modelleme feature'ının tamamında) benign'e karşı KS testi çalıştırıyor — model eğitiminden tamamen bağımsız, saf istatistiksel bir dağılım karşılaştırması. Amaç: "apache_bench'in kaçınılma sebebi, feature'ların hiç sinyal taşımaması mı, yoksa VAE'nin o sinyali reconstruction error'a çeviremiyor olması mı?" sorusuna kanıt toplamak.

Projedeki somut sonuç ve KS-yüksek/etki-küçük paradoksu:

feature	KS stat	p-değeri	mean shift (benign std)
orig_pkts_scaled	0.755	≈ 0	-0.44 sigma
resp_bytes_scaled	0.754	≈ 0	-0.68 sigma
duration_scaled	0.693	≈ 0	-0.42 sigma

13/18 feature'da $p < 0.05$ — yani istatistiksel olarak apache_bench "kesinlikle farklı" bir dağılımdan geliyor, ve en güçlü feature'larda $KS \geq 0.69$ gibi görünüşte yüksek bir değer var. **Ama** bu KS büyüklüğü ile "mean shift in benign std" (etki büyüklüğü) sütunu birbirini çelişkili gibi görünen şekilde tamamlıyor: en güçlü feature'larda bile ortalama kayması sadece ~ 0.4 - 0.7 benign standart sapması — yani apache_bench'in "merkezi", hâlâ benign'in normal aralığının **içinde**, kuyruğunda değil.

Bunun sebebi KS istatistiğinin doğası: apache_bench çok **dar, düşük varyanslı bir küme** (aynı sabit boyutlu HTTP GET isteği tekrar tekrar gönderiliyor — p5-p95 aralığı neredeyse tek bir noktaya çöküyor), bu yüzden onun ampirik CDF'i, benign'in çok daha geniş yayılan CDF'ine karşı **dikeyde keskin bir sıçrama** yapıyor — dar+yoğun bir dağılım, geniş bir dağılıma göre CDF farkını büyütür, merkezleri birbirine yakın olsa bile. Yani **KS büyük olabilir** **sıfır bir grup "iğne ucu gibi dar" olduğu için, merkezler arasındaki mesafe küçük kalsa bile**.

Bu paradoksun pratik sonucu: reconstruction error, her feature'daki **sapmaların karesinin toplamıdır** (benign-fit bir manifold'a göre). Bir nokta benign dağılımının normal aralığının **içinde** duruyorsa (mean shift küçükse), CDF'i ne kadar keskin sıçrarsa sıçrarsın, reconstruction error düşük kalır — model onu "tanıdık" görür. Bu tam olarak apache_bench'te olan: KS=0.75 gibi "istatistiksel olarak kesin farklı" bir sinyal var, ama VAE bunu yakalayamıyor çünkü reconstruction error mekanizması CDF şeklini değil, **mesafeyi** (öklid benzeri bir büyüklüğü) cezalandırıyor.

Karşılaştırma (portscan/slowloris): Bu paradoksun tersini portscan/slowloris'te görüyoruz — onların KS istatistikleri apache_bench'inkiyle benzer aralıkta (~0.6-1.0) ama mean shift'leri **onlarca-yüzlerce** benign standart sapması (örn. slowloris `byte_ratio_scaled` +1355 sigma, portscan `conn_state_SF` -31 sigma). Yani onlarda KS **ve** etki büyüklüğü birlikte yüksek — bu yüzden VAE onları neredeyse kusursuz yakalıyor (ROC-AUC 0.998-1.000).

Güçlü/zayıf yönleri: KS istatistiği tek başına "**ne kadar farklı**" sorusuna değil, "**farklı mı, değil mi**" sorusuna cevap verir — büyüklük (effect size) için ayrıca bir ölçüye (bu projede "mean shift in benign std") ihtiyaç var. Sadece KS'ye bakıp "apache_bench ayrışabilir görünüyor" demek, tam olarak bu projenin düştüğü ve sonra düzelttiği bir tuzaktı (`findings.md`, Bölüm 1: "on paper, apache_bench looks separable"). KS her zaman bir effect-size ölçüsüyle birlikte okunmalı.

Diğer metriklerle ilişkisi/gerilimi: KS, model-öncesi (model-agnostic) bir dağılım testi — ROC-AUC/PR-AUC'un aksine hiçbir modelin çıktısına bağlı değildir, saf feature-seviyesinde çalışır. Bu projede KS ile ROC-AUC arasındaki **uyumsuzluk** (KS yüksek görünürken ROC-AUC düşük kalması) tam olarak bu bölümdeki paradoksun kanıtı ve `04_apache_bench_diagnostics/` 'in ana bulgusudur. Ayrıca projede bir **temporal (zamansal) KS testi** de var: flow'lar arası varış süresi (IAT) için KS=0.7097 ($p \approx 0$), ama burada asıl önemli olan yine KS değil, etki büyüklüğü — apache_bench'in medyan IAT'ı (0.00092s) benign'inkinden (2.18s) **~2364 kat** kısa, yani birkaç merteye büyüklüğünde bir fark (mevcut tekil-flow feature'ların 0.4-0.7 sigma'lık farklarıyla kıyaslanamayacak kadar büyük) — ama bu, VAE'ye eklenmiş/retrain edilmiş bir feature değil, sadece bir hipotez testi; retrain ile doğrulanmadı.

7. Bootstrap Confidence Interval (Bootstrap Güven Aralığı)

Formül/tanım:

Elimizde n tane bağımsız ölçüm varsa (bu projede: bir kontaminasyon seviyesindeki $n = 20$ seed'in her birinin PR-AUC'u), bootstrap yöntemi şöyle çalışır:

1. Bu n değerden, **yerine koyarak (with replacement)** n tanesini rastgele çek (bazı değerler birden fazla, bazıları hiç seçilmeyebilir).
2. Bu yeniden-örnekleme ortalamasını hesapla.
3. 1-2 adımını binlerce kez (bu projede **10.000 kez**) tekrarla — elde edilen binlerce "yeniden-örnekleme ortalaması", ortalamanın olası dağılımını yaklaşık olarak simüle eder.
4. Bu dağılımın 2.5. ve 97.5. percentile'ları arasındaki aralık, **%95 güven aralığıdır (CI)**.

İki grubu (örn. bir kontaminasyon seviyesi vs. %0 temiz baseline) karşılaştırmak için: her iki gruptan bağımsız olarak yeniden-örnekle, **farkı** (level_mean – baseline_mean) hesapla, bu farkın dağılımının %95 CI'ını al. Eğer bu aralık **0'ı içermiyorsa**, fark istatistiksel olarak anlamlı sayılır.

Ne soruyor: "Gözlemlediğim ortalama fark (örn. %1 kontaminasyon PR-AUC'u ile %0'ınki arasındaki fark), gerçek mi, yoksa sadece seed-seçiminin şansı mı?"

Projede nasıl kullanıldı: `phase3_vae/05_contamination_sweep/bootstrap_significance.py`, 9 kontaminasyon seviyesinin (%0/1/2/4/8/12/14.33/19.30/21.29) her birinin 20-seed'lik PR-AUC dağılımını %0 (clean-only) baseline'a karşı bootstrap ile karşılaştırıyor:

Kontam.	mean	diff_from_0%	%95 CI	Anlamlı mı?
0%	0.716	— (baseline)	—	—
1%	0.697	-0.018	[-0.029, -0.009]	Evet
2%	0.691	-0.024	[-0.030, -0.018]	Evet
4%	0.676	-0.040	[-0.053, -0.029]	Evet
8%	0.639	-0.077	[-0.105, -0.052]	Evet
12%	0.634	-0.081	[-0.121, -0.047]	Evet

Sonuç: **hiçbir kontaminasyon seviyesinin CI'ı 0'ı kapsamıyor** — train setine karşı saldırı flow'larının oranı ne kadar küçük olursa olsun (%1 dahil), VAE'nin PR-AUC'u istatistiksel olarak anlamlı biçimde düşüyor. Ayrıca CI genişliği kontaminasyon arttıkça büyüyor (%1'de genişlik 0.020, %12'de 0.074) — bu, yüksek kontaminasyon seviyelerinde seed'ler arası değişkenliğin (bkz. aşağıdaki bimodal örnek) arttığının doğrudan bir kanıtı.

Neden 5-seed yeterli değildi — bimodal dağılım örneği: Bu projenin en çarpıcı metodolojik derslerinden biri burada yatıyor. Deney başta her kontaminasyon seviyesinde sadece **5 seed** ile çalıştırılmıştı ve ilk sonuç şuydu: "%12 kontaminasyon (mean PR-AUC=0.683), %8'den (mean=0.641) belirgin biçimde daha iyi — bir 'toparlanma' var." Seviye 20 seed'e çıkarılınca bu yorum **tamamen çöktü**:

- 20-seed'lik ölçümde %12'nin gerçek mean'i **0.634** çıktı — yani eski 5-seed'lik %12 örneklemin (seed 0-4), **şansla**, o seviyedeki 20 seed'in "kötü kümesine" (PR-AUC<0.58) hiç düşmemiş 5 iyi seed'i yakalamıştı.
- Kök neden: %8 ve üstündeki her kontaminasyon seviyesinde, seed'lerin PR-AUC dağılımı **bimodal** (iki kümeli) — seed'lerin ~%80-85'i sıkı bir "iyi" kümede (~0.63-0.72) toplanırken, ~%15-20'si çok daha düşük bir "kötü" kümeye (~0.40-0.55) düşüyor (sıradan eğitim instabilitesi, posterior-collapse değil — latent boyut aktivasyonu kontrol edilerek doğrulandı). Örnek: %8 seviyesinde 20 seed'in 4'ü (PR-AUC: 0.478, 0.551, 0.562, 0.566) kötü kümeye düşüyor, %12'de yine 4/20 (0.402, 0.469, 0.520, 0.550).
- **Bimodal bir dağılımdan yalnızca 5 örnek çekildiğinde**, hepsinin "iyi" kümeye düşme olasılığı $0.8^5 \approx \%33$ 'tür — yani bu üçte-bir ihtimal, "olağanüstü şanslı" değil, oldukça **olağan** bir sonuçtur. Bu da demek oluyor ki 5-seed'lik bir örneklemin, gerçek dağılımın bimodal doğasını hiç yansıtmadan, tesadüfen tamamen "iyi kümeden" oluşması sürpriz değil — ve tam olarak %12'de olan da buydu.
- Bunun genel dersi: **küçük n ile düşük std, "düşük gerçek değişkenlik" in değil, "şanslı örnekleme"nin işareti olabilir**. Standart sapma tek başına, örneklem küçükken güvenilir bir belirsizlik göstergesi değildir — bootstrap CI (veya doğrudan daha fazla seed) olmadan "bu sonuç kararlı" demek yanıltıcı olabilir.

Güçlü/zayıf yönleri: Bootstrap, dağılımın şekli hakkında (normal mi, bimodal mı) hiçbir varsayım yapmaz — bu, tam olarak bu projedeki bimodal PR-AUC dağılımı için onu doğru araç yapan özelliktir (klasik bir t-testi normal dağılım varsayar ve burada yanıltıcı olurdu). Zayıf yönü: hâlâ elinizdeki n örneklemin **popülasyonu temsil ettiğini** varsayar — 20 seed'in kendisi de yine sınırlı bir örneklem, "gerçek" kötü-seed oranının %10-20 mi yoksa başka bir sayı mı olduğu konusunda 20 seed de mutlak kesinlik vermez, sadece 5 seed'den çok daha güvenilir bir tahmin verir.

Diğer metriklerle ilişkisi/gerilimi: Bootstrap CI, tekil bir PR-AUC sayısına (Bölüm 5) **belirsizlik** ekler — "PR-AUC=0.634" yerine "PR-AUC=0.634, %95 CI=[0.55, 0.72]" gibi bir ifadeye izin verir, bu da iki seviyeyi (örn. %8 vs %12) karşılaştırırken "gerçekten farklılar mı, yoksa gürültü mü" sorusuna nicel bir cevap sağlar — nitekim projedeki "%12 toparlanması" iddiasının **yanlış** olduğu tam olarak bu araçla (20-seed karşılaştırması + median/trimmed-mean) ortaya çıkarıldı.

8. Reconstruction Error (Yeniden İnşa Hatası)

Diğer metriklerden farkı: Bu bölüme kadar anlatılan her metrik (Recall, Precision, F1, ROC-AUC, PR-AUC) **değerlendirme** metrikleridir — modelin çıktısını gerçek etiketlerle (attack/benign) karşılaştırırlar. Reconstruction error ise bir **değerlendirme metriği değil, modelin kendisinin ürettiği bir skordur** — VAE/Dense Autoencoder'ın çekirdek çıktısı. Diğer tüm metrikler, aslında bu tek skorun üzerine kuruludur: reconstruction error → sabit bir eşikle (threshold_95) ikiye bölünür (saldırı / benign) → o ikili karardan TP/FP/TN/FN → yukarıdaki tüm metrikler hesaplanır.

Formül/tanım: Autoencoder, bir flow'un x feature vektörünü önce düşük boyutlu bir gizli (latent) temsile sıkıştırır, sonra bu temsilden $\hat{x}$ 'i yeniden inşa etmeye çalışır. Reconstruction error, genelde kare farkların toplamıdır:

$$\text{Error}(x) = \sum_i (x_i - \hat{x}_i)^2$$

(VAE'de buna ayrıca bir KL-divergence terimi, β ile ağırlıklanmış, eklenir — eğitim kaybı $\mathcal{L} = \text{Recon.Error} + \beta \cdot D_{KL}$, ama değerlendirme/anomali skoru olarak kullanılan çoğunlukla saf reconstruction error'dur.)

Neden anomaly detection'da kullanılır: Model **sadece benign trafik üzerinde** eğitiliyor — hiç saldırı etiketi görmeden. Model, benign trafiğin "tipik" desenlerini bir latent uzayda sıkıştırıp geri açmayı öğreniyor. Bir flow benign'in öğrendiği normal desenlere uyuyorsa, model onu kolayca (düşük hatayla) yeniden inşa edebilir. Bir flow bu normal desenlere **uymuyorsa**, model onu sıkıştırıp-açarken "zorlanır" ve hata büyür — bu büyüme, "anormallik"ın dolaylı bir göstergesi olarak kullanılır. Bu, **denetimsiz (unsupervised)** bir yaklaşımdır: saldırı örneklerine hiç ihtiyaç duymadan (ki gerçek dünyada yeni saldırı türleri zaten etiketli örnek olarak elde bulunmaz) anomali tespiti yapılabilir.

"Skor" olması ve threshold ile binary karara çevrilmesi: Reconstruction error, kendi başına sürekli (continuous) bir sayıdır — 0'dan sonsuza kadar herhangi bir değer alabilir, doğal bir "saldırı/benign" sınırı yoktur. Bunu bir karara çevirmek için bir **eşik (threshold)** gerekir. Bu proje threshold_95 kullanıyor: held-out bir benign validasyon setinin hata dağılımının 95. percentile'ı — yani "eğer bu flow, normal benign trafiğin en 'kötü' %5'inden bile daha yüksek hata veriyorsa, alarm ver" kuralı. Bu seçim doğrudan Precision/Recall dengesini belirler (Bölüm 1-2'deki trade-off): eşik düşürülürse (örn. 90. percentile) recall artar ama FP (dolayısıyla precision düşüşü) artar; eşik yükseltirirse (örn. 99. percentile) tam tersi olur.

Projedeki somut sonuç: 04_apache_bench_diagnostics/findings.md, Bölüm 3 — VAE reconstruction error, gruplara göre:

grup	n	ortalama hata	std hata	thr95'te işaretlenen %
benign	6821	0.0605	0.659	%4.6
apache_bench	1487	5.746	37.88	%2.6
portscan+slowloris	1623	56.910	50.280	%100.0

Bu tablo, ROC-AUC/PR-AUC'un neden apache_bench'te düşük çıktığını en ham haliyle gösteriyor: apache_bench'in **ortalama** hatası benign'inkinin ~95 katı (5.746 vs 0.0605) olsa da, dağılımlar o kadar geniş örtüşüyor ki threshold_95'te işaretlenen oran (%2.6) **benign'in kendi false-positive oranından (%4.6) bile düşük** — yani apache_bench'in çoğu flow'u benign'in normal hata aralığının içinde kalıyor. Buna karşılık portscan+slowloris grubunda hata o kadar büyük (ortalama 56.910 — apache_bench'in ~9900 katı) ki hepsi (%100) eşiği aşıyor. Bu, Bölüm 6'daki KS-paradoksunun (dağılım şekli farklı ama merkez benign'e yakın) reconstruction-error uzayındaki doğrudan yansımasıdır.

Güçlü/zayıf yönleri: Reconstruction error'ün gücü, hiç etiketli saldırı verisi gerektirmemesi — yeni/görülmemiş saldırı türlerine karşı bile prensipte çalışabilir (supervised bir sınıflandırıcının aksine). Zayıf yönü, tam olarak apache_bench örneğinde görülüyor: eğer bir saldırının **tekil-flow düzeyinde** görünümü normalden çok az sapıyorsa (apache_bench'in stereotipik ama "sıradan görünümlü" HTTP istekleri gibi), model onu hiç yakalayamaz — hata, modelin öğrendiği feature uzayının **ötesindeki** (örn. istek sıklığı, ardışık flow'lar arası zaman deseni gibi çapraz-flow) sinyalleri hiç göremez. [04_apache_bench_diagnostics/findings.md](#) Bölüm 5-6, bunun için bir düzeltme **hipotezi** öneriyor (pencere-bazlı istek hızı/eşzamanlılık feature'ları) ama bunu retrain ile doğrulamıyor — mevcut haliyle bu hâlâ açık bir sınırlama.

Diğer metriklerle ilişkisi/gerilimi: Reconstruction error, bu dokümandaki tüm diğer metriklerin **girdisidir**: threshold_95 ile ikili karara çevrilince Recall/Precision/F1 (Bölüm 1-3) hesaplanabilir hale gelir; eşiği tüm olası değerlerde tararsanız ROC/PR eğrileri (Bölüm 4-5) ortaya çıkar. Contamination sweep deneyinde de ([phase3_vae/05_contamination_sweep/](#)), train setine saldırı flow'u karışıkça modelin bu flow'ları da "normal" olarak öğrenmeye başlaması — yani reconstruction error dağılımının benign/attack için birbirine yaklaşması — PR-AUC'daki düşüşün (Bölüm 5/7) altında yatan doğrudan mekanizmadır.

Özet Tablo: Hangi Metriğe Ne Zaman Bakmalı

Senaryo	Önerilen metrik(ler)	Neden
Saldırı kaçırma riski en kritikse (örn. veri sızıntısı, kritik altyapı)	Recall	Sadece FN'i (kaçırılan saldırı) cezalandırır; en "güvenlik odaklı" metrik
Yanlış alarm maliyeti yüksekse (analist zamanı kısıtlı, alert fatigue riski)	Precision	Sadece FP'yi (yanlış alarm) cezalandırır
İki hatayı da dengeli, tek sayıda özetlemek gerekiyorsa	F1	Precision ve recall'ün ikisi de düşükse yüksek çıkmaz (harmonik ortalama)
Dengesiz veri setinde (attack azınlıkta) genel model gücü	PR-AUC	TN'nin büyüklüğüyle seyrelmez, gerçek precision-recall dengesini yansıtır (bkz. Bölüm 5)
Dengeli veya yaklaşık dengeli veri setinde, eşikten bağımsız genel ayırt etme gücü	ROC-AUC	Yorumlanması sezgisel (0.5=rastgele, 1.0=mükemmel), ama dengesiz veride yanıltıcı olabilir
"Bu feature/sinyal, bu saldırı türü için prensipte var mı?" (model-öncesi keşif)	KS istatistiği + etki büyüklüğü (mean shift)	KS tek başına yanıltıcı olabilir (Bölüm 6 paradoksu); ikisi birlikte gerekir
"Bu sonuç gerçek mi, yoksa şanslı bir örneklemin ürünü mü?" (az sayıda seed/deneme ile)	Bootstrap CI	Dağılım şekli hakkında varsayım yapmaz, bimodal durumlarda bile güvenilir (Bölüm 7)
Modelin ham "anormallik" sinyalini görmek, threshold'dan önce	Reconstruction Error	Diğer tüm metriklerin girdisi; dağılımına bakmak, threshold seçiminin ve F1/recall sonuçlarının "neden"ini gösterir
Model karşılaştırması (örn. VAE vs Dense v1) yaparken	ROC-AUC + PR-AUC birlikte, eşiğe bağlı sonuç için F1	Tek bir metrik yeterli değil — eşikten bağımsız (AUC'lar) ve eşiğe bağlı (F1) resmin ikisi de gerekli
Kontaminasyon/veri kalitesi gibi bir deneysel değişkenin etkisini test ederken	PR-AUC + Bootstrap CI, mutlaka ≥ 20 seed	5 seed, bimodal seed-to-seed varyansı yakalamaya yetmeyebilir (Bölüm 7)

Genel kural: Tek bir metriğe güvenmeyin. Bu projenin en önemli metodolojik dersi tam olarak bu — apache_bench'in ROC-AUC'u (0.58) tek başına "zayıf ama belki kullanılabilir" izlenimi verebilirdi; PR-AUC (0.21, taban çizgisine yakın) ve reconstruction-error dağılımı (Bölüm 8) bunun "pratikte işe yaramaz" olduğunu netleştirdi. Aynı şekilde %12 kontaminasyon seviyesinin 5-seed mean'i (0.683) "toparlanma" gibi görünüyordu; 20 seed + bootstrap CI bunun bir örnekleme şanssı olduğunu ortaya çıkardı. Metrikler birbirini doğrulamak veya çürütmek için birlikte okunmalı.